

华中科技大学

2025

系统能力培养 课程实验报告

题 目:	指令模拟器
专 业:	计算机科学与技术
班 级:	CS2207
学 号:	U202215561
姓 名:	瞿明睿
电 话:	13956929603
邮 件:	3021018823@qq. com
完成日期:	2026-1-18



目 录

1	课程实验概述	1
2	实验过程	2
2.1	PA0	2
2.2	PA1	3
2.2.1	简易调试实现	3
2.2.2	表达式求值实现	3
2.2.3	监视点功能实现	4
2.2.4	实现测试	5
2.3	PA2	9
2.3.1	dummy 和更多指令	9
2.3.2	运行所有 cputest 程序	10
2.3.3	打字小游戏	10
2.3.4	测试结果	10
2.4	PA3	13
2.4.1	实现自陷操作_yield() 及其过程	13
2.4.2	实现用户程序的加载和系统调用, 支撑 TRM 程序的运行	15
2.4.3	运行 pal 并展示批处理系统	17
2.4.4	测试	19
2.5	PA4	20
2.5.1	上下文切换与多道程序	20
2.5.2	虚拟内存管理与多道程序系统	20
2.5.3	抢占式分时多任务系统实验报告	23
2.5.4	测试	25
3	总结与问题	30

1 课程实验概述

本课程实验旨在通过从零构建一个功能完备的 RISC-V 指令集模拟器，深入探究“程序在计算机系统中运行”的底层机理。实验涵盖了从基础指令解码到复杂操作系统支撑的关键环节，包括 异常处理机制、精简文件系统、虚拟内存管理以及抢占式多任务调度 等核心功能 。通过这一过程，不仅实现了软硬件接口的协同设计，更在实践中梳理了从数字逻辑、组成原理到操作系统的全层级系统栈知识，显著提升了大规模系统级代码的编写与调试能力 。

具体任务上来说：

在代码框架中实现一个简化的 RISC-V 模拟器

可解释执行 RISC-V 执行代码

支持输入输出设备

支持异常流处理

支持精简操作系统---支持文件系统

支持虚存管理

支持进程分时调度

最终在模拟器上运行“仙剑奇侠传”，探究“程序在计算机上运行”的机理，掌握计算机软硬协同的机制，进一步加深对计算机分层系统栈的理解，梳理大学 3 年所学的全部理论知识，提升计算机系统能力。

2 实验过程

2.1 PA0

本人使用 windows11 下 wsl 实现 linux 环境，使用 Ubuntu-24.04 版本实现实验过程，其中工具版本问题因为版本问题做了实验配置和包的修改，在最后提到。

Updating APT package information

Now you can tell `apt` to retrieve software information from the sources:

```
apt-get update
```

However, you will receive an error message:

```
E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)
E: Unable to lock directory /var/lib/apt/lists/
```

This is because `apt-get` requires superuser privilege to run. Run the command with superuser privilege. Since it requires Internet accessing, it may cost some time to finish.

Installing tools for PAs

The following tools are necessary for PAs:

```
apt-get install build-essential # build-essential packages, include binary utilities, gcc, make
apt-get install man             # on-line reference manual
apt-get install gdb             # GNU debugger
apt-get install git             # reversion control system
apt-get install libreadline-dev # a library to use compile the project later
apt-get install libsdl2-dev     # a library to use compile the project later
apt-get install libc6-dev-i386  # a library to use compile the project later
apt-get install qemu-system     # QEMU
```

The usage of these tools is explained later.

只需要按顺序安装图上的工具即可，安装源在华中科技大学校园网环境下可以直接 ping 到，所以不需要换。

安装完成库，熟悉 linux 操作即可，不需要额外工作，PA0 到此上完成。

有关包名的问题会在最后写。

2.2 PA1

PA1.1: 实现单步执行, 打印寄存器状态, 扫描内存

PA1.2: 实现算术表达式求值

PA1.3: 实现所有要求, 提交完整的实验报告

2.2.1 简易调试实现

寄存器显示与查找在项目文件为 `nemu/src/isa/riscv32/reg.c` 文件中的 `isa_reg_display()` 函数:

通过遍历 `regsl[]` 数组, 使用 `reg_l(i)` 寄存器, 输出十六进制和十进制格式

```
uint32_t isa_reg_str2val(const char *s, bool *success) {
    int i;
    if (s[0] == '$') s++; // 跳过$前缀
    for (i = 0; i < 32; i++) {
        if (strcmp(regsl[i], s) == 0) {
            *success = true;
            return reg_l(i);
        }
    }
    *success = false;
    return 0;
}
```

单步调试实现在文件 `nemu/src/monitor/debug/ui.c`, 实现的 `si` 命令, 在 `nemu` 的 `bash` 中调用 `si [N]` 调用 `cpu_exec(N)` 函数, `info` 命令显示寄存器状态或监视点信息, `x` 命令内存扫描使用 `vaddr_read(addr, 4)` 读取内存

```
static int cmd_x(char *args) {
    char *n_str = strtok(args, " ");
    int n = atoi(n_str);

    char *expr_str = strtok(NULL, " ");
    uint32_t addr;
    sscanf(expr_str, "%x", &addr);

    for (int i = 0; i < n; i++) {
        uint32_t data = vaddr_read(addr + i * 4, 4);
        printf("0x%08x: 0x%08x\n", addr + i * 4, data);
    }
    return 0;
}
```

2.2.2 表达式求值实现

我们可以利用栈的方法实现，也就是数据结构应用的几个典型例子，但是这是一种在底层设计上不太好的方法，应该使用编译原理的思路去实现，同时这个命令并不要求完全完美的求值实现，我们可以参考编译原理中对于 token 的处理和语法树的构建思路完成这个功能。同时因为不太要求性能和时间问题，那么对于求具体的值，我们可以使用递归的方法求解。

文件在 nemu/src/monitor/debug/expr.c

```
enum { //补充字段
    TK_NOTYPE = 256, TK_EQ,
    TK_PLUS, TK_MINUS, TK_MULT, TK_DIV,
    TK_LPAREN, TK_RPAREN,
    TK_DEC_NUM, TK_HEX_NUM,
    TK_REG, TK_DEREF, TK_NEQ, TK_AND
};
```

正则表达式规则：

```
static struct rule {
    char *regex;
    int token_type;
} rules[] = { //源程序已经完成一部分，补充如下
    {" +", TK_NOTYPE},           // 空格
    {"\\+", '+'},               // 加号
    {"-", TK_MINUS},            // 减号
    {"\\*", '*'},               // 乘号/解引用
    {"/", '/'},                 // 除号
    {"==", TK_EQ},              // 等于
    {"!=", TK_NEQ},             // 不等于
    {"&&", TK_AND},             // 逻辑与
    {"\\(", '('},               // 左括号
    {"\\)", ')'},               // 右括号
    {"0x[0-9a-fA-F]+", TK_HEX_NUM}, // 十六进制数
    {"[0-9]+", TK_DEC_NUM},      // 十进制数
    {"\\$[a-zA-Z0-9]+", TK_REG}, // 寄存器
};
```

运算过程中使用几个函数如下，也是补充即可

check_parentheses(p, q) - 检查括号匹配：

find_dominant_op(p, q) - 找主运算符：

其中优先级定义如：0: &&; 1: ==, !=; 2: +, -; 3: *, /; 4: !, *

eval(p, q, success) - 递归求值函数：

2.2.3 监视点功能实现

相关文件:

nemu/include/monitor/watchpoint.h

nemu/src/monitor/debug/watchpoint.c

nemu/src/monitor/cpu-exec.c

监视点数据结构

```
typedef struct watchpoint {
    int NO;
    struct watchpoint *next;
    char expr[128];      // 监视的表达式
    uint32_t old_val;    // 旧值
    uint32_t new_val;    // 新值
} WP;
```

监视点池管理

```
new_wp(expr_str) - 分配监视点
free_wp(NO) - 释放监视点
check_watchpoints() - 检查监视点
```

将这三个函数集成到 CPU 执行循环方法如下

1. 使用固定大小的对象池 (NR_WP = 32) 维护两个链表: head (活跃监视点链表) 和 free_ (空闲监视点链表), 初始化时所有监视点都在空闲链表中。

2. 新建监视点时, 从空闲链表取一个 WP 保存表达式字符串, 加入到 head 链表末尾, 删除监视点时从 head 链表找到对应编号的监视点, 从 head 链表移除, 放回 free_ 空闲链表

3. 检查监视点时, 遍历所有活跃监视点, 对每个表达式求值得到新值, 与 old_val 比较, 如果不同则打印监视点信息, 更新 old_val 为新值, 返回 true 表示有变化

4. cpu 则在每条指令执行后、下一条指令执行前检查监视点, 如果检测到值变化则暂停 CPU 执行。

这样就完成监视点的功能。

2.2.4 实现测试

(nemu) si

```

[src/monitor/monitor.c,28,welcome] Build time: 17:07:10, Dec 29 2025
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) si
80100000: b7 02 00 80          lui  0x80000,t0
(nemu) si
80100004: 23 a0 02 00          sw   0(t0),$0
(nemu) si
80100008: 03 a5 02 00          lw   0(t0),a0
(nemu) si
8010000c: 6b 00 00 00          nemu trap
nemu: HIT GOOD TRAP at pc = 0x8010000c

```

(nemu) info r

```

[src/monitor/cpu-exec.c,29,monitor_statistic] total guest instructions = 4
(nemu) info r
$0      0x00000000      0
ra      0x00000000      0
sp      0x00000000      0
gp      0x00000000      0
tp      0x00000000      0
t0      0x80000000     -2147483648
t1      0x00000000      0
t2      0x00000000      0
s0      0x00000000      0
s1      0x00000000      0
a0      0x00000000      0
a1      0x00000000      0
a2      0x00000000      0
a3      0x00000000      0
a4      0x00000000      0
a5      0x00000000      0
a6      0x00000000      0
a7      0x00000000      0
s2      0x00000000      0
s3      0x00000000      0

```

(nemu) x 2 80100000

```

[src/monitor/monitor.c,28,welcome] Build time: 17:07:10, Dec 29 2025
Welcome to riscv32-NEMU!
For help, type "help"
(nemu) x 2 80100000
0x80100000: 0x800002b7
0x80100004: 0x0002a023

```

(nemu) p 1+2*3


```

0x00100004: 0x00020029
(nemu) p 1+2*3
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,92,make_token] match rules[1] = "+" at position 1 with len 1: +
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 2 with len 1: 2
[src/monitor/debug/expr.c,92,make_token] match rules[3] = "*" at position 3 with len 1: *
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 4 with len 1: 3
7 (0x00000007)

```

(nemu) p (1+2)*(3+4)

```

0x00000007:
(nemu) p (1+2)*(3+4)
[src/monitor/debug/expr.c,92,make_token] match rules[8] = "(" at position 0 with len 1: (
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 1 with len 1: 1
[src/monitor/debug/expr.c,92,make_token] match rules[1] = "+" at position 2 with len 1: +
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 3 with len 1: 2
[src/monitor/debug/expr.c,92,make_token] match rules[9] = ")" at position 4 with len 1: )
[src/monitor/debug/expr.c,92,make_token] match rules[3] = "*" at position 5 with len 1: *
[src/monitor/debug/expr.c,92,make_token] match rules[8] = "(" at position 6 with len 1: (
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 7 with len 1: 3
[src/monitor/debug/expr.c,92,make_token] match rules[1] = "+" at position 8 with len 1: +
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 9 with len 1: 4
[src/monitor/debug/expr.c,92,make_token] match rules[9] = ")" at position 10 with len 1: )
21 (0x00000015)
(nemu)

```

(nemu) p 10-5-2

```

(nemu) p 10-5-2
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 0 with len 2: 10
[src/monitor/debug/expr.c,92,make_token] match rules[2] = "-" at position 2 with len 1: -
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 3 with len 1: 5
[src/monitor/debug/expr.c,92,make_token] match rules[2] = "-" at position 4 with len 1: -
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 5 with len 1: 2
3 (0x00000003)

```

(nemu) p 5==5

```

(nemu) p 5==5
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 0 with len 1: 5
[src/monitor/debug/expr.c,92,make_token] match rules[5] = "==" at position 1 with len 2: ==
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 3 with len 1: 5
1 (0x00000001)

```

(nemu) p 5!=4

```

(nemu) p 5!=4
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 0 with len 1: 5
[src/monitor/debug/expr.c,92,make_token] match rules[6] = "!=" at position 1 with len 2: !=
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 3 with len 1: 4
1 (0x00000001)
(nemu)

```

(nemu) p 1&&0

```

(nemu) p 1&&0
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,92,make_token] match rules[7] = "&&" at position 1 with len 2: &&
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 3 with len 1: 0
0 (0x00000000)

```

(nemu) p \$sp+4

```
(nemu) p $sp+4
[src/monitor/debug/expr.c,92,make_token] match rules[12] = "\\$[a-zA-Z0-9]+" at position 0 with len 3: $sp
[src/monitor/debug/expr.c,92,make_token] match rules[1] = "\\+" at position 3 with len 1: +
[src/monitor/debug/expr.c,92,make_token] match rules[11] = "[0-9]+" at position 4 with len 1: 4
4 (0x00000004)
```

(nemu) p *0x100000

```
(nemu) p *0x80100000
[src/monitor/debug/expr.c,92,make_token] match rules[3] = "*" at position 0 with len 1: *
[src/monitor/debug/expr.c,92,make_token] match rules[10] = "0x[0-9a-fA-F]+" at position 1 with len 10: 0x80100000
2147484343 (0x800002b7)
```

监视点测试

(nemu) w \$a0

```
(nemu) w $a0
Set watchpoint 0: $a0
Old value = 0x00000000
```

(nemu) info w

```
(nemu) w $a0
Set watchpoint 0: $a0
Old value = 0x00000000
(nemu) info w
```

NO	Expr	Value
0	\$a0	0x00000000

(nemu)

(nemu) c #程序运行，当监视值变化时自动停止

```
(nemu) w $t0
Set watchpoint 0: $t0
Old value = 0x00000000
(nemu) c
[src/monitor/debug/expr.c,92,make_token] match rules[12] = "\\$[a-zA-Z0-9]+" at position 0 with len 3: $t0

Hardware watchpoint 0: $t0
Old value = 0x00000000
New value = 0x80000000
```

(nemu) d 0

```
(nemu) d 0
Watchpoint 0 deleted
```

2.3 PA2

PA2.1: 在 NEMU 中运行第一个 C 程序 dummy

PA2.2: 实现更多的指令，在 NEMU 中运行所有 cputest

PA2.3: 运行打字小游戏

2.3.1 dummy 和更多指令

RISC-V 指令集采用六种基本格式：

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0	
funct7				rs2		rs1	funct3		rd			opcode		R-type	
imm[11:0]						rs1	funct3		rd			opcode		I-type	
imm[11:5]				rs2		rs1	funct3		imm[4:0]			opcode		S-type	
imm[12]	imm[10:5]			rs2		rs1	funct3		imm[4:1]	imm[11]	opcode		B-type		
imm[31:12]										rd			opcode		U-type
imm[20]	imm[10:1]			imm[11]		imm[19:12]			rd			opcode		J-type	

在 nemu 中实现现有指令之外的指令，我们需要加上更多的筛选字段（其实是在做软件上的编码器）

在 /home/sanstoolow/ics2019/nemu/src/isa/riscv32/include/isa-def.h 中定义指令解码联合体：如下

```
typedef union {
    struct {
        uint32_t opcode : 7;
        uint32_t rd : 5;
        uint32_t funct3 : 3;
        uint32_t rs1 : 5;
        uint32_t rs2 : 5;
        uint32_t funct7 : 7;
    };
    uint32_t val;
} ISADecodeInfo;
```

这里对于指令 decode 的字段映射表在 nemu/src/isa/riscv32/exec/exec.c 中实现，定义为如下结构

```
static OpcodeEntry opcode_table [32] = {
    /* b00 */ IDEX(ld, load), EMPTY, EMPTY, EMPTY, IDEX(I, alu_i), IDEX(U,
    auipc), EMPTY, EMPTY,
    /* b01 */ IDEX(st, store), EMPTY, EMPTY, EMPTY, IDEX(R, alu_r),
    IDEX(U, lui), EMPTY, EMPTY,
```

```

/* b10 */ EMPTY, EMPTY, EMPTY, EMPTY, EMPTY, EMPTY, EMPTY, EMPTY,
/* b11 */ IDEX(B, branch), EMPTY, EX(nemu_trap), IDEX(J, jal), EMPTY,
EMPTY, EMPTY, EMPTY,
};

```

通过反编译查明，我们需要实现的指令

Lui、auipc、Jal、Jalr、beq/bne/blt/bge、add/addi、sub、mul R-type 均为在 compute 中实现的关键字。

2.3.2 运行所有 cputest 程序

实现完整的 RV32IMA 指令集，使所有 cputest 测试程序通过。

加载/存储指令 在 exec 中添加 load_table 和 store_table

```

static OpcodeEntry load_table [8] = {
    EXW(ld, 1), EXW(ld, 2), EXW(ld, 4), EMPTY, EXW(ldu, 1), EXW(ldu, 2),
    EMPTY, EMPTY
};
static OpcodeEntry store_table [8] = {
    EXW(st, 1), EXW(st, 2), EXW(st, 4), EMPTY, EMPTY, EMPTY, EMPTY, EMPTY
};

```

乘除法指令：mul、mulh、Mulhsu、Mulhu、Div、Divu、Rem、remu

比较指令：slt、slti、sltu、sltiu：

在 compute 等依次实现指令内容即可，和之前做出 gdb 实现类似，和组原课设也相似，做硬编码即可。

2.3.3 打字小游戏

IOE 通过 AM 接口连接 NEMU 设备和应用程序，实现流程如下

应用程序→AM 接口 (_io_read/_io_write)→NEMU 设备驱动→SDL2 图形库

MMIO 表规范如下

设备	地址	说明
键盘	0xa1000060	i8042 数据端口
定时器	0xa1000048	RTC 时间戳
屏幕信息	0xa1000100	宽高
同步信号	0xa1000104	屏幕刷新触发
帧缓冲	0xa0000000	显存区域

2.3.4 测试结果

测试全部的 cputest 如下

```
sanstoolow@DESKTOP-CNBR5K5:~/ics2019/nexus-am/tests/cputest$ make ARCH=riscv32-nemu run | grep -r "TRAP"
make: [Makefile:13: Makefile.bit] Broken pipe (ignored)
make: [Makefile:13: Makefile.shuixianhua] Broken pipe (ignored)
make: [Makefile:13: Makefile.recursion] Broken pipe (ignored)
make: [Makefile:13: Makefile.quick-sort] Broken pipe (ignored)
make: [Makefile:13: Makefile.prime] Broken pipe (ignored)
make: [Makefile:13: Makefile.add-longlong] Broken pipe (ignored)
make: [Makefile:13: Makefile.load-store] Broken pipe (ignored)
make: [Makefile:13: Makefile.max] Broken pipe (ignored)
make: [Makefile:13: Makefile.dummy] Broken pipe (ignored)
make: [Makefile:13: Makefile.hello-str] Broken pipe (ignored)
make: [Makefile:13: Makefile.fib] Broken pipe (ignored)
make: [Makefile:13: Makefile.wanshu] Broken pipe (ignored)
make: [Makefile:13: Makefile.switch] Broken pipe (ignored)
make: [Makefile:13: Makefile.bubble-sort] Broken pipe (ignored)
make: [Makefile:13: Makefile.sub-longlong] Broken pipe (ignored)
make: [Makefile:13: Makefile.leap-year] Broken pipe (ignored)
make: *** [Makefile:7: all] Broken pipe
● sanstoolow@DESKTOP-CNBR5K5:~/ics2019/nexus-am/tests/cputest$ { make ARCH=riscv32-nemu run 2>&1 || true; } | grep TRAP
nemu: HIT GOOD TRAP at pc = 0x8010014c
nemu: HIT GOOD TRAP at pc = 0x8010014c
nemu: HIT GOOD TRAP at pc = 0x8010009c
nemu: HIT GOOD TRAP at pc = 0x80100130
nemu: HIT GOOD TRAP at pc = 0x8010017c
nemu: HIT GOOD TRAP at pc = 0x80100120
nemu: HIT GOOD TRAP at pc = 0x80100138
nemu: HIT GOOD TRAP at pc = 0x80100104
nemu: HIT GOOD TRAP at pc = 0x80100154
nemu: HIT GOOD TRAP at pc = 0x80100118
nemu: HIT GOOD TRAP at pc = 0x80100184
nemu: HIT GOOD TRAP at pc = 0x8010013c
nemu: HIT GOOD TRAP at pc = 0x80100118
nemu: HIT GOOD TRAP at pc = 0x801001bc
nemu: HIT GOOD TRAP at pc = 0x80100158
nemu: HIT GOOD TRAP at pc = 0x80100120
nemu: HIT GOOD TRAP at pc = 0x80100158
nemu: HIT GOOD TRAP at pc = 0x8010022c
nemu: HIT GOOD TRAP at pc = 0x8010012c
nemu: HIT GOOD TRAP at pc = 0x8010024c
nemu: HIT GOOD TRAP at pc = 0x801003d0
nemu: HIT GOOD TRAP at pc = 0x801000e0
nemu: HIT GOOD TRAP at pc = 0x80100148
nemu: HIT GOOD TRAP at pc = 0x801001d4
nemu: HIT GOOD TRAP at pc = 0x80100128
nemu: HIT GOOD TRAP at pc = 0x80100030
nemu: HIT GOOD TRAP at pc = 0x80100108
nemu: HIT GOOD TRAP at pc = 0x801000c0
nemu: HIT GOOD TRAP at pc = 0x801000ec
nemu: HIT GOOD TRAP at pc = 0x8010010c
nemu: HIT GOOD TRAP at pc = 0x80100130
nemu: HIT GOOD TRAP at pc = 0x80100148
nemu: HIT GOOD TRAP at pc = 0x8010010c
```

可以看到全部通过。

打字小游戏如下

```
13  @make -s -f $@ ARCH=${ARCH} $(MAKECMDGOALS)
14  @rm -f Makefile.$*
15
16  # cancel rules included by $(AM_HORE)/Makefile.check
17  image: ;
18  default $(MAKECMDGOALS): all ;
19
20  clean:
21  rm -rf Makefile.* build/
22
23  latest:
24
25  问题 输出 调试控制台 窗口 终端
26
27  sanstoolow@DESKTOP-CHRSKS:~/ics2019/nexus-am/apps/typing$ make ARCH=riscv32-nemu run
28  + O3COPY -> build/typing-riscv32-nemu.bin
29  make -C /home/sanstoolow/ics2019/nemu ISA=riscv32 run ARGS="-b -l /home/sanstoolow/ics2019/nemu
30  u.bin"
31  make[1]: Entering directory '/home/sanstoolow/ics2019/nemu'
32  Building riscv32-nemu
33  make -C /home/sanstoolow/ics2019/nemu/tools/qemu-diff
34  make[2]: Entering directory '/home/sanstoolow/ics2019/nemu/tools/qemu-diff'
35  make[2]: Nothing to be done for 'app'.
36  make[2]: Leaving directory '/home/sanstoolow/ics2019/nemu/tools/qemu-diff'
37  ./build/riscv32-nemu -b -l /home/sanstoolow/ics2019/nexus-am/apps/typing/build/nemu-log.txt /h
38  tools/qemu-diff/build/riscv32-qemu-so
39  [src/monitor/monitor.c:48,load_img] The image is /home/sanstoolow/ics2019/nexus-am/apps/typing/
40  [src/memory/memory.c:16,register_pmem] Add 'pmem' at [0x00000000, 0x07ffffff]
41  [src/device/io/port-io.c:16,add_pio_map] Add port-io map 'serial' at [0x0000003f8, 0x0000003f8]
42  [src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'serial' at [0x100003f8, 0x100003f8]
43  [src/device/io/port-io.c:16,add_pio_map] Add port-io map 'rtc' at [0x000000048, 0x00000004b]
44  [src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'rtc' at [0x10000048, 0x1000004b]
45  [src/device/io/port-io.c:16,add_pio_map] Add port-io map 'screen' at [0x000000100, 0x000000107]
46  [src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'screen' at [0x1000100, 0x1000107]
47  [src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'vmem' at [0x00000000, 0x0007ffff]
48  [src/device/io/port-io.c:16,add_pio_map] Add port-io map 'keyboard' at [0x00000000, 0x000000063]
49  [src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'keyboard' at [0x10000000, 0x10000063]
50  [src/monitor/monitor.c:25,welcome] Debug: OFF
51  [src/monitor/monitor.c:28,welcome] Build time: 14:26:37, Jan 18 2026
52  Welcome to riscv32-NEMU!
53  For help, type "help"
54  []
```



2.4 PA3

PA3.1:实现自陷操作_yield()及其过程

PA3.2:实现用户程序的加载和系统调用,支撑 TRM 程序的运行

PA3.3:运行仙剑奇侠传并展示批处理系统

2.4.1 实现自陷操作_yield()及其过程

PA3.1 需要实现: ecall, 保存上下文以及事件分发

先添加 CSR 寄存器定义, 在 reg.h 中添加寄存器

```
{ vaddr_t pc;
  vaddr_t stvec;    // 异常入口地址
  vaddr_t sepc;     // 异常返回地址
  uint32_t sstatus; // 处理器状态
  uint32_t scause;  // 异常原因
} CPU_state;
```

Ecall 通过 raise_intr 在 intr.c 中实现异常触发:

```
void raise_intr(uint32_t NO, vaddr_t epc) {
    cpu.sepc = epc;    // 保存触发异常的 PC
    cpu.scause = NO;   // 设置异常号

    vaddr_t tvec = cpu.stvec;
    decinfo.jump_pc = tvec; // 跳转到异常入口
    decinfo.is_jump = 1;
}
```

补充 nemu 在 ecall/sret/CSR 指令的完善调用处理

在 system.c 中实现指令:

```
case 0x0:
    if (decinfo.isa.instr.funct7 == 0) {
        // ecall 指令, 触发异常号 11 的自陷
        raise_intr(11, cpu.pc);
    }
else if (decinfo.isa.instr.funct7 == 0x08) {
    // sret
    decinfo.jump_pc = cpu.sepc + 4; // riscv32 需要 +4 跳过 ecall
    decinfo.is_jump = 1;
}
case 0x1: { // csrrw
    uint32_t csr_num = decinfo.isa.instr.csr;
    switch(csr_num) {
        case 0x100: /* sstatus */ break;
        case 0x105: /* stvec */ break;
```

```

    case 0x141: /* sepc */ break;
    case 0x142: /* scause */ break;
}
}

```

修改 AM 部分，补充上下文的问题，在 riscv32-nemu.h 定义 _Context，与 trap.S 的栈布局一致即可

```

struct _Context {
    uintptr_t gpr[32]; // 31 个通用寄存器 + sp_original
    uintptr_t scause; // 偏移 32*4
    uintptr_t sstatus; // 偏移 33*4
    uintptr_t sepc; // 偏移 34*4
    struct _AddressSpace *as;
};

```

其实这里 X2 没有保存，因为 NEMU 没有使用传统思路去做这个系统，而是通过调用拓展栈，不断+context 的大小拓展，所以在之后的实现也没有任何必要使用 x2，也给上下文这里实现提供了逻辑。

事件分发 cte.c 中识别自陷事件：

```

Context* __am_irq_handle(_Context *c) {
    _Event ev = {0};
    switch (c->scause) {
        case 11: // ecall 异常号
            ev.event = _EVENT_YIELD;
            break;
        default:
            ev.event = _EVENT_ERROR;
            break;
    }
    next = user_handler(ev, c);
    return next ? next : c;
}

```

事件处理在 irq.c

```

static _Context* do_event(_Event e, _Context* c) {
    switch (e.event) {
        case _EVENT_YIELD:
            Log("Get _EVENT_YIELD, will return to user program...");
            break;
        default:
            panic("Unhandled event ID = %d", e.event);
    }
}

```



```

return c; // 返回上下文指针，用于恢复执行
}

```

yield 执行流程如下

```

main()
→ _yield()
→ ecall 指令
→ NEMU: raise_intr(11, pc)
→ 保存 sepc=pc, scause=11
→ 跳转到 stvec (trap.S)
→ trap.S: __am_asm_trap
→ 保存 31 个通用寄存器到栈（像之前说的 x2 是忽略的）
→ 保存 scause, sstatus, sepc 到栈
→ 调用 __am_irq_handle(sp)
→ 识别 scause=11 为 _EVENT_YIELD
→ 调用 user_handler (do_event)
→ 输出日志，返回 c
→ 返回 next
→ 恢复 sstatus, sepc
→ 恢复通用寄存器
→ sret
→ NEMU: pc = sepc + 4
→ 返回 _yield()
→ 继续执行到 panic()

```

2.4.2 实现用户程序的加载和系统调用，支撑 TRM 程序的运行

实现用户程序加载和系统调用：ELF 程序加载器，系统调用机制（SYS_exit, SYS_yield, SYS_write, SYS_brk），标准输出和堆区管理

在 CTE 区分 yield 与 syscall，利用 a7 寄存器值区分：-1 为 yield，0 为系统调用。

```

case 11: // ecall
    if ((int32_t)c->gpr[17] == -1) { // a7 == -1
        ev.event = _EVENT_YIELD;
    } else {
        ev.event = _EVENT_SYSCALL;
    }
    break;

```

ELF 加载 loader.c，解析 ELF header，加载 PT_LOAD 段到指定内存地址。

```

static uintptr_t loader(PCB *pcb, const char *filename) {
    Elf_Ehdr ehdr;
    ramdisk_read(&ehdr, 0, sizeof(Elf_Ehdr));
    assert(*(uint32_t *)ehdr.e_ident == 0x464c457f); // 验证 ELF 魔数

```

```

Elf_Phdr phdr;
for (int i = 0; i < ehdr.e_phnum; i++) {
    ramdisk_read(&phdr, ehdr.e_phoff + i * ehdr.e_phentsize,
sizeof(Elf_Phdr));
    if (phdr.p_type == PT_LOAD) {
        ramdisk_read((void *)phdr.p_vaddr, phdr.p_offset, phdr.p_filesz);
        memset((void *) (phdr.p_vaddr + phdr.p_filesz), 0, phdr.p_memsz -
phdr.p_filesz);
    }
}
return ehdr.e_entry;
}

```

系统调用处理 `irq.c`, 添加 `_EVENT_SYSCALL` 事件

```

case _EVENT_SYSCALL:
    do_syscall(c);
    break;

```

`syscall.c` 实现四个系统调用:

```

switch (a[0]) {
case SYS_exit: _halt(a[1]); break;
case SYS_yield: _yield(); c->GPRx = 0; break;
case SYS_write:
    if (a[1] == 1 || a[1] == 2) { // stdout/stderr
        char *buf = (char *)a[2];
        for (size_t i = 0; i < a[3]; i++) _putc(buf[i]);
        c->GPRx = a[3];
    } else c->GPRx = -1;
    break;
case SYS_brk: c->GPRx = 0; break; // 总是成功
}

```

用户态 `libos`, `nanos.c` 封装 `SYS_write` 系统调用, 调用 `SYS_brk`

```

int _write(int fd, void *buf, size_t count) {
    return _syscall_(SYS_write, fd, (intptr_t)buf, count);
}

extern char _end;
void *_sbrk(intptr_t increment) {
    static char *program_break = NULL;
    if (program_break == NULL) program_break = &_amp;_end;
    char *old_break = program_break;
    if (_syscall_(SYS_brk, (intptr_t)(program_break + increment), 0, 0)
== 0) {
        program_break += increment;
    }
}

```

```

    return old_break;
}
return (void *)-1;
}

```

加载用户程序 `proc.c` 在 `init_proc()` 中调用 `naive_uoload` 加载 `dummy` 程序:

```

void init_proc() {
    switch_boot_pcb();
    Log("Initializing processes...");
    naive_uoload(NULL, NULL);
}

```

2.4.3 运行 `pal` 并展示批处理系统

实现简易文件系统、设备抽象和批处理系统，完成“一切皆文件”的操作系统设计理念。

VFS 实现 `fs.c`

```

typedef struct {
    char *name;
    size_t size;
    size_t disk_offset;
    size_t open_offset;    // 新增: 文件读写位置指针
    ReadFn read;           // 读函数指针
    WriteFn write;         // 写函数指针
} Finfo;

```

实现文件记录表, 实现文件操作, 如 `fs_open`, `fs_read`, `fs_write`, `fs_lseek`, `fs_close` 等

```

static Finfo file_table[] = {
    {"stdin", 0, 0, 0, invalid_read, invalid_write},
    {"stdout", 0, 0, 0, invalid_read, serial_write},
    {"stderr", 0, 0, 0, invalid_read, serial_write},

    {"dev/events", 0, 0, 0, events_read, invalid_write},
    {"proc/dispinfo", 128, 0, 0, dispinfo_read, invalid_write},
    {"dev/fb", 0, 0, 0, invalid_read, fb_write},
    {"dev/fbsync", 0, 0, 0, invalid_read, fbsync_write},
    {"dev/tty", 0, 0, 0, invalid_read, serial_write},

    #include "files.h"
};

```

设备 I/O 在 `device.c`, 串口输出

```

size_t serial_write(const void *buf, size_t offset, size_t len) {

```

```

const char *p = (const char *)buf;
for (size_t i = 0; i < len; i++) {
    _putc(p[i]); // 逐字符输出到串口
}
return len;
}

```

事件输入（键盘+定时器），屏幕信息和串口等类似上方

系统调用扩展 syscall.c

```

_Context* do_syscall(_Context *c) {
    uintptr_t a[4];
    a[0] = c->GPR1; // 调用号
    a[1] = c->GPR2; // 参数 1
    a[2] = c->GPR3; // 参数 2
    a[3] = c->GPR4; // 参数 3

    switch (a[0]) {
        case SYS_open:
            c->GPRx = fs_open((const char *)a[1], a[2], a[3]);
            break;
        case SYS_read:
            c->GPRx = fs_read(a[1], (void *)a[2], a[3]);
            break;
        case SYS_write:
            c->GPRx = fs_write(a[1], (const void *)a[2], a[3]);
            break;
        case SYS_close:
            c->GPRx = fs_close(a[1]);
            break;
        case SYS_lseek:
            c->GPRx = fs_lseek(a[1], a[2], a[3]);
            break;
        case SYS_execve:
            naive_uload(NULL, (const char *)a[1]);
            break;
        case SYS_exit:
            naive_uload(NULL, "/bin/init"); // 批处理：重启菜单
            break;
        // 其他调用
    }
    return NULL;
}

```

用户库 nanos.c 更新使用 syscall 即可，loader.c 更新如下

```

static uintptr_t loader(PCB *pcb, const char *filename) {

```

```

int fd = fs_open(filename, 0, 0); // 打开 ELF 文件

Elf_Ehdr ehdr;
fs_read(fd, &ehdr, sizeof(Elf_Ehdr));
assert(*(uint32_t *)ehdr.e_ident == 0x464c457f); // 验证 ELF 魔数

// 加载所有 LOAD 段
Elf_Phdr phdr;
for (int i = 0; i < ehdr.e_phnum; i++) {
    fs_lseek(fd, ehdr.e_phoff + i * ehdr.e_phentsize, SEEK_SET);
    fs_read(fd, &phdr, sizeof(Elf_Phdr));

    if (phdr.p_type == PT_LOAD) {
        fs_lseek(fd, phdr.p_offset, SEEK_SET);
        fs_read(fd, (void *)phdr.p_vaddr, phdr.p_filesz);
        memset((void *) (phdr.p_vaddr + phdr.p_filesz), 0,
                phdr.p_memsz - phdr.p_filesz);
    }
}

fs_close(fd);
return ehdr.e_entry;
}

```

2.4.4 测试

这里测试内容和 PA4 很重复，省略。

2.5 PA4

PA4.1: 实现基本的多道程序系统

PA4.2: 实现支持虚存管理的多道程序系统

PA4.3: 实现抢占式分时多任务系统

2.5.1 上下文切换与多道程序

本阶段的目标是实现上下文切换，支持多个线程的并发执行。在 AM 层实现上下文的创建和切换。在 Nanos-lite 层实现简单的 schedule

在 trap.S 中看到，__am_irq_handle 返回新进程的上下文结构指针，即新进程的栈顶，将 a0 赋值给 sp，就完成栈切换

```
mv a0, sp
jal __am_irq_handle
mv sp, a0    // [关键修改] 将 a0 (新上下文地址) 赋值给 sp, 完成栈切换
lw t1, OFFSET_STATUS(sp) // 从新栈恢复状态
// ... 恢复其他寄存器
sret
```

cte.c 中完成_kcontext

在给定的栈空间末尾构造一个_Context 结构，使得它可以被 trap.S 正确恢复。

sepc 设为线程入口函数 entry

sstatus 设为 MPP=11, MPIE=1, 确保线程运行在特权模式且开中断。

gpr[10] (a0): 传递参数 arg`。

返回指向构造好的_Context 指针。

实现最简单的轮转调度策略即可，将当前上下文 prev 保存到 current->cp，在 pcb[0]和 pcb[1]之间交替切换 current 指针。返回 current->cp 作为下一个要运行的上下文。

2.5.2 虚拟内存管理与多道程序系统

PA4.2 的核心目标是在 PA4.1 上下文切换的基础上，实现虚拟内存管理和完整的多道程序系统。在 AM 层实现虚拟地址到物理地址的映射(_vme_init, _map, _protect)，修改 loader.c 使用虚拟地址加载用户程序，实现 SYS_brk 系统调用的动态内存分配支持堆管理，确保多个用户进程能够拥有独立的虚拟地址空间并正确切换

vme.c 虚拟内存初始化_vme_init

为内核空间建立恒等映射（虚拟地址 = 物理地址），包括：

物理内存区域 0x80000000 ~ 0x88000000（128MB）

MMIO 区域 0xa0000000 ~ 0xa0001000

```
// 填充内核页表
for (i = 0; i < NR_KSEG_MAP; i++) {
    uint32_t pdir_idx = (uintptr_t)segments[i].start / (PGSIZE * NR_PTE);
    uint32_t pdir_idx_end = (uintptr_t)segments[i].end / (PGSIZE *
NR_PTE);
    for (; pdir_idx < pdir_idx_end; pdir_idx++) {
        // 填充 PDE
        kpdirs[pdir_idx] = ((uintptr_t)ptab >> PGSHFT << 10) | PTE_V;
        // 填充 PTE（恒等映射）
        PTE pte = (PGADDR(pdir_idx, 0, 0) >> PGSHFT << 10) | PTE_V | PTE_R
| PTE_W | PTE_X;
        // ...
    }
}
set_satp(kpdirs); // 启用分页
```

创建用户地址空间_protect，为每个用户进程分配独立的页目录，并复制内核映射

```
int _protect(_AddressSpace *as) {
    PDE *updir = (PDE*)(pgalloc_usr(1)); // 分配新页目录
    as->ptr = updir;
    // 复制内核映射到用户页目录
    for (int i = 0; i < NR_PDE; i++) {
        updir[i] = kpdirs[i];
    }
    return 0;
}
```

页表映射_map，建立虚拟地址到物理地址的映射

```
int _map(_AddressSpace *as, void *va, void *pa, int prot) {
    PDE *pdir = (PDE *)as->ptr;
    uint32_t pdx = PDX(va);
    uint32_t ptx = PTX(va);
    // 如果页表不存在，分配新页表
    if (!(pdir[pdx] & PTE_V)) {
        PTE *new_ptab = (PTE *)pgalloc_usr(1);
        for (int i = 0; i < NR_PTE; i++) {
            new_ptab[i] = 0;
        }
    }
}
```

```

    }
    pdir[pdx] = ((uintptr_t)new_ptab >> 12 << 10) | PTE_V;
}
// 填充页表项
PTE *ptab = (PTE *)PTE_ADDR(pdir[pdx]);
ptab[ptx] = ((uintptr_t)pa >> 12 << 10) | PTE_V | PTE_R | PTE_W |
PTE_X;
return 0;
}

```

地址空间切换__am_switch, 在上下文切换时同步切换页表基址寄存器

```

void __am_switch(_Context *c) {
    if (vme_enable && c->as != NULL) {
        printf("[VME] __am_switch: as=%p, pdir=%p\n", c->as, c->as->ptr);
        set_satp(c->as->ptr); // 切换 SATP
        cur_as = c->as;
    }
}

```

Loader.c 中 context_upload 创建地址空间, 分配用户栈, 按页加载 ELF 段到虚拟地址, 初始化 max_brk

```

void context_upload(PCB *pcb, const char *filename) {
    _protect(&pcb->as);
    pcb->max_brk = 0;
    //分配用户栈
    uintptr_t ustack_top = 0x80000000u;
    for (int i = 0; i < 8; i++) {
        void *pa = new_page(1);
        _map(&pcb->as, (void *) (ustack_top - (i + 1) * PGSIZE), pa, 0);
    }
    uintptr_t entry = loader(pcb, filename);
    //创建用户上下文
}

```

堆管理 mm_brk, 动态扩展用户进程的堆区:

```

int mm_brk(uintptr_t brk, intptr_t increment) {
    uintptr_t new_brk = brk + increment;

    if (new_brk > current->max_brk) {
        uintptr_t old_end = PGROUNDUP(current->max_brk);
        uintptr_t new_end = PGROUNDUP(new_brk);
        // 分配并映射新页面
        for (uintptr_t va = old_end; va < new_end; va += PGSIZE) {
            void *pa = new_page(1);
            _map(&current->as, (void *)va, pa, 0);
        }
    }
}

```



```

    }
    current->max_brk = new_brk;
}
return 0;
}

```

2.5.3 抢占式分时多任务系统实验报告

虚拟内存的基础上，实现基于时钟中断的抢占式调度机制，使系统无需依赖用户程序主动调用_yield()，能够自动切换进程，实现真正的分时多任务系统。

NEMU: device_update()->timer_intr()->dev_raise_intr()->cpu.INTR = true

RISC-V: exec_once() 检查->isa_query_intr()->检测 INTR && SIE->raise_intr(IRQ_TIMER) ->跳转到 stvec->触发上下文切换

RISC-V 规范 sstatus.SIE 位控制全局中断开关: SIE=1 允许接收中断; SIE=0 屏蔽所有中断。通过 csrsi sstatus, 0x2 和 csrci sstatus, 0x2 指令原子地设置此位。

事件分发如下

```

时钟中断 (scause=0x80000005)
→ __am_irq_handle() 识别为 _EVENT_IRQ_TIMER
→ do_event() 调用 schedule()
→ 切换到下一个进程

```

在 CPU_state 结构体中添加中断字段

```

typedef struct {
    //现有字段
    uint32_t sstatus;
    uint32_t scause;
    uint32_t satp;
    bool INTR;          // 新增: 中断引脚
} CPU_state;

```

NEMU 时钟中断机制 intr.c 中，时钟到期时，timer_intr 调用此函数，cpu.INTR = true 模拟硬件中断信号

```

void dev_raise_intr() {
    extern CPU_state cpu;
    cpu.INTR = true; // 设置中断引脚
}

bool isa_query_intr(void) {

```

```

// 检查: INTR 引脚高电平 && SIE 开中断
if (cpu.INTR && (cpu.sstatus & SSTATUS_SIE)) {
    cpu.INTR = false; // 清除中断信号
    raise_intr(IRQ_TIMER, cpu.pc); // 触发时钟中断异常
    return true;
}
return false;
}

```

AM 中断使能控制, 实现_intr_read 和_intr_write

```

int _intr_read() {
    uint32_t sstatus;
    asm volatile("csrr %0, sstatus" : "=r"(sstatus));
    return (sstatus & 0x2) != 0; // 返回 SIE 位状态
}

void _intr_write(int enable) {
    if (enable) {
        asm volatile("csrsi sstatus, 0x2"); // 设置 SIE
    } else {
        asm volatile("csrci sstatus, 0x2"); // 清除 SIE
    }
}

```

拓展 NEMU CSR 指令, 实现 csrsi 和 csrci 指令, 添加 funct3=0x6(csrrsi) 和 funct3=0x7(csrrci) 的支持:

```

case 0x6: { // csrrsi (CSR read and set bits immediate)
    uint32_t csr_num = decinfo.isa.instr.csr;
    uint32_t zimm = decinfo.isa.instr.rs1; // 5-bit immediate
    uint32_t rd = decinfo.isa.instr.rd;
    // 读取 CSR 当前值
    uint32_t csr_val = 0;
    switch(csr_num) {
        case 0x100: csr_val = cpu.sstatus; break;
        // ... 其他 CSR
    }
    // 写回到目标寄存器
    if (rd != 0) {
        rtl_sr(rd, &csr_val, 4);
    }
    // 如果立即数非零, 则设置对应位
    if (zimm != 0) {
        switch(csr_num) {
            case 0x100: cpu.sstatus |= zimm; break;
            // ... 其他 CSR
        }
    }
}

```

```

    }
    print_asm("csrrsi %s, 0x%x, %d", reg_name(rd, 4), csr_num, zimm);
    break;
}
case 0x7: { // csrrci (CSR read and clear bits immediate)
    // 类似 csrrsi, 但使用 &= ~zimm 清除位
    // ... 完整代码见源文件
}

```

事件分发给__am_irq_handle 中识别时钟中断:

```

_Context* __am_irq_handle(_Context *c) {
    __am_get_cur_as(c);
    _Context *next = c;
    if (user_handler) {
        _Event ev = {0};
        switch (c->scause) {
            case 11: // ecall
                if ((int32_t)c->gpr[17] == -1) {
                    ev.event = _EVENT_YIELD;
                } else {
                    ev.event = _EVENT_SYSCALL;
                }
                break;
            case 0x80000005: // 时钟中断 (IRQ_TIMER)
                ev.event = _EVENT_IRQ_TIMER;
                break;
            default:
                ev.event = _EVENT_ERROR;
                break;
        }
        next = user_handler(ev, c);
        if (next == NULL) {
            next = c;
        }
    }
    __am_switch(next);
    return next;
}

```

0x80000005 是时钟中断的标准编码, 最高位为 1 表示中断 (非异常), 低 31 位的值 5 表示 Supervisor Timer Interrupt

2.5.4 测试

以下是调试过程

```
[src/monitor/monitor.c,48,load_img] The image is /home/sanstorm/ics2019/nanos-lite/build/nanos-lite-riscv32-nemu.bin
[src/memory/memory.c,16,register_pmem] Add 'pmem' at [0x00000000, 0x7fffffff]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'serial' at [0x000003f8, 0x000003f8]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'serial' at [0x010003f8, 0x010003f8]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'rtc' at [0x00000048, 0x0000004b]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'rtc' at [0x01000048, 0x0100004b]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'screen' at [0x00000100, 0x00000107]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'screen' at [0x01000100, 0x01000107]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'vmm' at [0x00000000, 0x007fffff]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'keyboard' at [0x00000000, 0x00000003]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'keyboard' at [0x01000000, 0x01000003]
[src/monitor/monitor.c,20,welcome] Debug: ON
[src/monitor/monitor.c,21,welcome] If debug mode is on, A log file will be generated to record every instruction NEMU executes. This may lead to a large log file. If it is not necessary, you can turn it off in include/common.h.
[src/monitor/monitor.c,28,welcome] Build time: 01:39:52, Jan 4 2026
Welcome to riscv32-NEMU!
For help, type "help"
[/home/sanstorm/ics2019/nanos-lite/src/main.c,14,main] 'Hello World!' from Nanos-lite
[/home/sanstorm/ics2019/nanos-lite/src/main.c,15,main] Build time: 15:48:30, Jan 7 2026
[/home/sanstorm/ics2019/nanos-lite/src/ramdisk.c,27,init_ramdisk] ramdisk info: start = 0x, end = 0x, size = -2146428231 bytes
[/home/sanstorm/ics2019/nanos-lite/src/device.c,35,init_device] Initializing devices...
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,19,init_irq] Initializing interrupt/exception handler...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,25,init_proc] Initializing processes...
[/home/sanstorm/ics2019/nanos-lite/src/loader.c,48,naive_uload] Jump to entry = 0x
[src/isa/riscv32/intr.c,23,raise_intr] Exception: cause=11, epc=0x8300025c, stvec=0x00100068
NEMU: HIT BAD TRAP at pc = 0x0010006c
[src/monitor/cpu-exec.c,20,monitor_statistic] total guest instructions = 121338
nemu@1: leaving directory "/home/sanstorm/ics2019/nemu"
```

这是第一次检测到 yield

```
/build/riscv32-nemu -b -l /home/sanstorm/ics2019/nanos-lite/build/nemu.log.txt /home/sanstorm/ics2019/nanos-lite/build/nanos-lite-riscv32-nemu.bin -d /home/sanstorm/ics2019/nemu/tools/qemu-diff/build/riscv32-qemu-so
[src/monitor/monitor.c,48,load_img] The image is /home/sanstorm/ics2019/nanos-lite/build/nanos-lite-riscv32-nemu.bin
[src/memory/memory.c,16,register_pmem] Add 'pmem' at [0x00000000, 0x7fffffff]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'serial' at [0x000003f8, 0x000003f8]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'serial' at [0x010003f8, 0x010003f8]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'rtc' at [0x00000048, 0x0000004b]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'rtc' at [0x01000048, 0x0100004b]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'screen' at [0x00000100, 0x00000107]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'screen' at [0x01000100, 0x01000107]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'vmm' at [0x00000000, 0x007fffff]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'vmm' at [0x01000000, 0x007fffff]
[src/device/io/port-io.c,16,add_pio_map] Add port-io map 'keyboard' at [0x00000000, 0x00000003]
[src/device/io/mmio.c,14,add_mmio_map] Add mmio map 'keyboard' at [0x01000000, 0x01000003]
[src/monitor/monitor.c,20,welcome] Debug: ON
[src/monitor/monitor.c,21,welcome] If debug mode is on, A log file will be generated to record every instruction NEMU executes. This may lead to a large log file. If it is not necessary, you can turn it off in include/common.h.
[src/monitor/monitor.c,28,welcome] Build time: 01:39:52, Jan 4 2026
Welcome to riscv32-NEMU!
For help, type "help"
[/home/sanstorm/ics2019/nanos-lite/src/main.c,14,main] 'Hello World!' from Nanos-lite
[/home/sanstorm/ics2019/nanos-lite/src/main.c,15,main] Build time: 15:42:46, Jan 7 2026
[/home/sanstorm/ics2019/nanos-lite/src/ramdisk.c,27,init_ramdisk] ramdisk info: start = 0x, end = 0x, size = -2146428231 bytes
[/home/sanstorm/ics2019/nanos-lite/src/device.c,35,init_device] Initializing devices...
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,19,init_irq] Initializing interrupt/exception handler...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,25,init_proc] Initializing processes...
[/home/sanstorm/ics2019/nanos-lite/src/loader.c,48,naive_uload] Jump to entry = 0x
[src/isa/riscv32/intr.c,23,raise_intr] Exception: cause=11, epc=0x8300025c, stvec=0x00100068
[/home/sanstorm/ics2019/nanos-lite/src/syscall.c,22,do_syscall] system panic: Unhandled syscall ID = 37
NEMU: HIT BAD TRAP at pc = 0x0010006c
```

这一张是正确处理但是缺少系统处理处理对应的 ID

```
[src/monitor/monitor.c,28,welcome] Build time: 02:28:45, Jan 12 2026
Welcome to riscv32-NEMU!
For help, type "help"
[/home/sanstorm/ics2019/nanos-lite/src/main.c,14,main] 'Hello World!' from Nanos-lite
[/home/sanstorm/ics2019/nanos-lite/src/main.c,15,main] Build time: 02:36:52, Jan 12 2026
[/home/sanstorm/ics2019/nanos-lite/src/ramdisk.c,27,init_ramdisk] ramdisk info: start = 0x80102ed4, end = 0x8010eaf46, size = 950386 bytes
[/home/sanstorm/ics2019/nanos-lite/src/device.c,35,init_device] Initializing devices...
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,19,init_irq] Initializing interrupt/exception handler...
kcontext: c=0x801fbf70, entry=0x8010096c, c->sepc=80100968 (will jump to 8010096c)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,25,init_proc] Initializing processes...
[/home/sanstorm/ics2019/nanos-lite/src/main.c,33,main] Finish initialization
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf70, sepc=80100968, mem[34]=80100968 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 1th time!
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf50, sepc=80101544, mem[34]=80101544 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 2th time!
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf50, sepc=80101544, mem[34]=80101544 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 3th time!
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf50, sepc=80101544, mem[34]=80101544 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 4th time!
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf50, sepc=80101544, mem[34]=80101544 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 5th time!
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf50, sepc=80101544, mem[34]=80101544 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 6th time!
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf50, sepc=80101544, mem[34]=80101544 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 7th time!
[/home/sanstorm/ics2019/nanos-lite/src/irq.c,6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,39,schedule] schedule: cp=0x801fbf50, sepc=80101544, mem[34]=80101544 (offset 136)
[/home/sanstorm/ics2019/nanos-lite/src/proc.c,16,hello_fun] Hello World from Nanos-lite for the 8th time!
```

这是用户线程和内核线程互相 hello 以及不断 yield 切换上下文

```

[src/monitor/monitor.c:28,welcome] Build time: 02:28:45, Jan 12 2026
Welcome to riscv32-NEMU!
For help, type "help"
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:14,main] 'Hello World!' from Nanos-lite
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:15,main] Build time: 02:44:39, Jan 12 2026
[/home/sanstoolow/ics2019/nanos-lite/src/ramdisk.c:27,init_ramdisk] ramdisk info: start = 0x80102e54, end = 0x801eae6, size = 950386 bytes
[/home/sanstoolow/ics2019/nanos-lite/src/device.c:59,init_device] Initializing devices...
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:19,init_irq] Initializing interrupt/exception handler...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:27,init_proc] Initializing processes...
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:33,main] Finish initialization
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-A] for the 1th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-B] for the 1th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-A] for the 2th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-B] for the 2th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-A] for the 3th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-B] for the 3th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-A] for the 4th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-B] for the 4th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-A] for the 5th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-B] for the 5th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-A] for the 6th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Thread-B] for the 6th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...

```

```

[/home/sanstoolow/ics2019/nanos-lite/src/main.c:14,main] 'Hello World!' from Nanos-lite
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:15,main] Build time: 16:44:42, Jan 15 2026
[/home/sanstoolow/ics2019/nanos-lite/src/ramdisk.c:27,init_ramdisk] ramdisk info: start = 0x80103488, end = 0x822596c5, size = 34955837 bytes
[/home/sanstoolow/ics2019/nanos-lite/src/device.c:59,init_device] Initializing devices...
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:19,init_irq] Initializing interrupt/exception handler...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:34,init_proc] Initializing processes...
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:33,main] Finish initialization
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite for the 1th time! (Thread-0)
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite for the 1th time! (Thread-1)
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite for the 2th time! (Thread-0)
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite for the 2th time! (Thread-1)
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite for the 3th time! (Thread-0)
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite for the 3th time! (Thread-1)
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...

```

```

sanstoolow@DESKTOP-CNBR5K5:~/ics2019/nanos-lite$ make ARCH=riscv32-nemu run | head -100
[src/monitor/monitor.c:48,load_img] The image is /home/sanstoolow/ics2019/nanos-lite/build/nanos-lite-riscv32-nemu.bin
[src/memory/memory.c:16,register_pmem] Add 'pmem' at [0x80000000, 0x87ffffff]
[src/device/io/port-io.c:16,add_pio_map] Add port-io map 'serial' at [0x000003f8, 0x000003f8]
[src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'serial' at [0xa10003f8, 0xa10003f8]
[src/device/io/port-io.c:16,add_pio_map] Add port-io map 'rtc' at [0x00000048, 0x0000004b]
[src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'rtc' at [0xa1000048, 0xa100004b]
[src/device/io/port-io.c:16,add_pio_map] Add port-io map 'screen' at [0x00000100, 0x00000107]
[src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'screen' at [0xa1000100, 0xa1000107]
[src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'vmem' at [0xa0000000, 0xa007ffff]
[src/device/io/port-io.c:16,add_pio_map] Add port-io map 'keyboard' at [0x00000060, 0x00000063]
[src/device/io/mmio.c:14,add_mmio_map] Add mmio map 'keyboard' at [0xa1000060, 0xa1000063]
[src/monitor/monitor.c:25,welcome] Debug: OFF
[src/monitor/monitor.c:28,welcome] Build time: 02:28:45, Jan 12 2026
Welcome to riscv32-NEMU!
For help, type "help"
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:14,main] 'Hello World!' from Nanos-lite
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:15,main] Build time: 02:48:35, Jan 12 2026
[/home/sanstoolow/ics2019/nanos-lite/src/ramdisk.c:27,init_ramdisk] ramdisk info: start = 0x80102f34, end = 0x801eafa6, size = 950386 bytes
[/home/sanstoolow/ics2019/nanos-lite/src/device.c:59,init_device] Initializing devices...
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:19,init_irq] Initializing interrupt/exception handler...
[/home/sanstoolow/ics2019/nanos-lite/src/loader.c:71,context_uload] context_uload: entry=830001d8, user_stack=0x88000000
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:30,init_proc] Initializing processes...
[/home/sanstoolow/ics2019/nanos-lite/src/main.c:33,main] Finish initialization
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
[/home/sanstoolow/ics2019/nanos-lite/src/proc.c:16,hello_fun] Hello World from Nanos-lite [Kernel-Thread] for the 1th time!
[/home/sanstoolow/ics2019/nanos-lite/src/irq.c:6,do_event] Get _EVENT_YIELD, will return to user program...
Hello World!
Hello World from Navy-apps for the 2th time!
Hello World from Navy-apps for the 3th time!
Hello World from Navy-apps for the 4th time!
Hello World from Navy-apps for the 5th time!
Hello World from Navy-apps for the 6th time!
Hello World from Navy-apps for the 7th time!
Hello World from Navy-apps for the 8th time!
Hello World from Navy-apps for the 9th time!
Hello World from Navy-apps for the 10th time!
Hello World from Navy-apps for the 11th time!
Hello World from Navy-apps for the 12th time!
Hello World from Navy-apps for the 13th time!

```

正确加载用户程序

```
_Context* schedule(_Context *prev) {
    // 保存当前上下文
    current->cp = prev;
    // 选择下一个进程/线程 (pcb[0] <-> pcb[1])
    current = (current == &pcb[0] ? &pcb[1] : &pcb[0]);
    // 返回新进程上下文
    return current->cp;
}

void init_proc() {
    switch_boot_pcb();

    Log("Initializing processes...");

    // load program here PA3
    // naive_upload(NULL, "/bin/text");
    // naive_upload(NULL, NULL);

    // 内核进程hello world pcb[0]
    _Area stack;
    stack.start = pcb[0].stack;
    stack.end = pcb[0].stack + sizeof(pcb[0].stack);
    pcb[0].cp = _kcontext(stack, hello_fun, "Thread-0");

    // 内核进程hello world pcb[1]
    stack.start = pcb[1].stack;
    stack.end = pcb[1].stack + sizeof(pcb[1].stack);
    pcb[1].cp = _kcontext(stack, hello_fun, "Thread-1");
}
```

这是多道程序的一个例子，这里的 schedule 是一个互相跳转的例子，基于可以得到前面内核线程互相 hello 的例子

```
25 // 保存当前进程的上下文
26 if (current->cp != NULL) {
27     current->cp = prev;
28 }
29
30 // 简单轮转: 0 -> 1 -> 2 -> 0
31 if (current == &pcb[0]) current = &pcb[1];
32 else if (current == &pcb[1]) current = &pcb[2];
33 else current = &pcb[0];
34
35 return current->cp;
36 }
37
38 void init_proc() {
39     switch_boot_pcb();
40 }

Hello World from Navy-apps for the 6211th time!
Hello World from Navy-apps for the 6212th time!
Hello World from Navy-apps for the 6213th time!
Hello World from Navy-apps for the 6214th time!
Hello World from Navy-apps for the 6215th time!
Hello World from Navy-apps for the 6216th time!
Hello World from Navy-apps for the 6217th time!
Hello World from Navy-apps for the 6218th time!
Hello World from Navy-apps for the 6219th time!
Hello World from Navy-apps for the 6220th time!
Hello World from Navy-apps for the 6221th time!
Hello World from Navy-apps for the 6222th time!
Hello World from Navy-apps for the 6223th time!
Hello World from Navy-apps for the 6224th time!
Hello World from Navy-apps for the 6225th time!
Hello World from Navy-apps for the 6226th time!
Hello World from Navy-apps for the 6227th time!
Hello World from Navy-apps for the 6228th time!
Hello World from Navy-apps for the 6229th time!
Hello World from Navy-apps for the 6230th time!
Hello World from Navy-apps for the 6231th time!

24 Context* schedule(Context *prev) {
25     // 保存当前进程的上下文
26     if (current->cp != NULL) {
27         current->cp = prev;
28     }
29
30     // 简单轮转: 0 -> 1 -> 2 -> 0
31     if (current == &pcb[0]) current = &pcb[1];
32     else if (current == &pcb[1]) current = &pcb[2];
33     else current = &pcb[0];
34
35     return current->cp;
36 }
37
38 void init_proc() {
39     switch_boot_pcb();
40 }

Hello World from Navy-apps for the 12290th time!
Hello World from Navy-apps for the 12291th time!
Hello World from Navy-apps for the 12292th time!
Hello World from Navy-apps for the 12293th time!
Hello World from Navy-apps for the 12294th time!
Hello World from Navy-apps for the 12295th time!
Hello World from Navy-apps for the 12296th time!
Hello World from Navy-apps for the 12297th time!
Hello World from Navy-apps for the 12298th time!
Hello World from Navy-apps for the 12299th time!
Hello World from Navy-apps for the 12300th time!
Hello World from Navy-apps for the 12301th time!
Hello World from Navy-apps for the 12302th time!
Hello World from Navy-apps for the 12303th time!
Hello World from Navy-apps for the 12304th time!
Hello World from Navy-apps for the 12305th time!
Hello World from Navy-apps for the 12306th time!
Hello World from Navy-apps for the 12307th time!
receive time event for the 12288th time: t -852300509
Hello World from Navy-apps for the 12308th time!
Hello World from Navy-apps for the 12309th time!
Hello World from Navy-apps for the 12310th time!
```

当实现了 PA4 终极目标，就可以在加载很多个用户程序的同时，写一个简单调度完成测试

3 总结与问题

在实验的执行过程中会有问题如下：

Ubuntu 在 24.04 新内核是取消了 `riscv-none-embed-*` 这样命名的包了，本人在尝试之后，使用 `gcc-riscv64-unknown-elf` 新包名正常安装交叉编译器是可以使用，但是需要使用符号链接。

```
sudo find / -type l -name "riscv" -ls 2>/dev/null 简化输出如下
/usr/bin/riscv-none-embed-ar -> /usr/bin/riscv64-unknown-elf-ar
/usr/bin/riscv-none-embed-objcopy -> /usr/bin/riscv64-unknown-elf-objcopy
/usr/bin/riscv-none-embed-objdump -> /usr/bin/riscv64-unknown-elf-objdump
/usr/bin/riscv-none-embed-ld -> /usr/bin/riscv64-unknown-elf-ld
/usr/bin/riscv-none-embed-gcc -> /usr/bin/riscv64-unknown-elf-gcc
/usr/bin/riscv-none-embed-g++ -> /usr/bin/riscv64-unknown-elf-g++
/home/sanstoolow/ics2019/nexus-am/am/include/arch/riscv32-navy.h
-> ../navy.h
```

通过编译器符号链接到的老的交叉编译上，在 `makefile` 中调用的 `riscv-none-embed` 均可以正常编译出结果，反编译也是正常的，但是新编译器参数有变化，在 `/home/sanstoolow/ics2019/nexus-am/am/arch/isa/riscv32.mk` 中需要修改编译的参数如下

原先为

```
COMMON_FLAGS := -fno-pic -march=rv32im -mabi=ilp32
```

修改为

```
COMMON_FLAGS := -fno-pic -march=rv32ima_zicsr -mabi=ilp32
```

因为如果使用新包，会生成 16 位压缩指令，然而 `nemu` 没有对应的处理。或者说如果对这个处理，那么在后续很多的指令都会有问题，特别是 PA4 在很多跳转的指令都会拓展导致一直无法正常继续下去，这里我们修改编译参数，生成符

合 nemu 要求的子集即可，正好简化处理即可，测试等主流的测试是可以正常通过的，也不算是问题，但是可以更新一下 wiki 等。

同时在 PA2.2 中，我有认为有必要出多一个测试样例：auipc 和 jalr 一起测试，这个问题在 PA3 中暴露了，我在 PA2 中的测试均可以 PASS 但是因为缺少 auipc 和 jalr 一起测试的内容，实际上我 auipc 实现有问题并没有体现出来，在具体实现中没有调用 cpu.pc 而是直接使用了 pc，导致在 PA3 中无法正确继续下去，这也是测试不够全面。

整体来说 PA 还是很有难度的，其实是调试出的问题太多，有很多内容是因为代码库庞大，文件很多，很难说可以用很短的时间完成实验，在很多功能的调试上也是经常需要查询文档，能够合理查找的自己需要的内容也是计算机人必修的。